

College Placement Drive and Student Application Tracker

A web-based system to digitize and streamline campus placement management using Python Flask and SQLite.

GET /api/placements/applications

```
# Presented By

name: ASHIKA.S
department: Information Technology
college: PSVPEC
stack: Python • Flask • SQLite
```

GETTING STARTED

Introduction

Problem Statement

Objective

ARCHITECTURE

System Comparison

Tech Stack

IMPLEMENTATION

Key Features

SQL Queries

Sample Output

File Structure

SUMMARY

How to Run

Future Scope

Conclusion

PROBLEM STATEMENT

Existing System

Placement officers maintain all student data and application statuses manually, leading to errors, delays, and missed opportunities.

01

Manual Record Keeping

Placement records are maintained manually with no standardized format.

02

Scattered Files

Student applications are stored in different files with no central access.

03

No Status Tracking

Difficult to track application status — Applied, Shortlisted, or Rejected.

04

Missed Deadlines

Students miss important placement deadlines due to lack of notifications.

05

No Centralized Database

There is no centralized database for quick search and report generation.

GETTING STARTED

- Introduction
- Problem Statement

Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

PROJECT GOALS

Objective

Build a centralized, web-based system to manage all placement-related data efficiently and accurately.

01 – Student Data

Store and manage all student information in a structured database.

02 – Company Details

Maintain company records including role, package, and drive date.

03 – Applications

Record and manage student placement applications with applied dates.

04 – Status Tracking

Track application status — Applied, Shortlisted, Interview, Selected, or Rejected.

05 – Search

Search and filter applications by student name or application status.

06 – Reports

Generate placement reports and counts for administrative use.

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

System Comparison

- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

ARCHITECTURE

Existing vs Proposed System

Moving from a fragmented manual process to a unified, digital, web-based platform.

FEATURE	EXISTING SYSTEM	PROPOSED SYSTEM
Record Keeping	Manual Records	Digital Database
Data Lookup	Difficult Search	Instant Search
Medium	Paper Based	Web Based
Data Integrity	Duplicate Records	Unique Constraints
Processing Speed	Time Consuming	Fast Processing

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison

Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

ARCHITECTURE

Technology Stack

A lightweight, open-source stack — chosen for simplicity, portability, and academic accessibility.

BE

Flask

Backend — Python micro web framework for routing and request handling

DB

SQLite

Database — Serverless relational DB, stores students, companies, applications

FE

HTML & CSS

Frontend — Jinja2 templates with responsive HTML & CSS styling

PY

Python 3.x

Language — Core programming language for all application logic

IDE

VS Code

Development environment with Python and Flask extensions

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

Key Features

- SQL Queries
- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

IMPLEMENTATION

Key Features

All core operations exposed through a simple, responsive web interface.

- Add new placement application
- View all applications
- Search by student name
- Filter by application status
- Track status history
- Record count and statistics
- Simple and responsive web UI

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features

SQL Queries

- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

IMPLEMENTATION

SQL Queries (Examples)

Core queries powering the application's data access layer.

Display all applications

```
SELECT * FROM
    Application
ORDER BY applied_date
    DESC;
```

Search by student name

```
SELECT * FROM
    Application
WHERE name LIKE
    '%Kamalika%';
```

Filter shortlisted students

```
SELECT * FROM
    StatusHistory
WHERE status =
    'Shortlisted';
```

Count total applications

```
SELECT
    COUNT(*)
FROM
    Application;
```


Project Overview

Full system diagram covering ER model, architecture, database tables, features, queries, and sample screens.

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries

Sample Output

- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

College Placement Drive and Student Application Tracker

1. Project Overview

Problem Statement:
Placement officers maintain student applications, company details and status updates manually which is time consuming and error prone.

Objective:

- To maintain student, company and application details.
- To track application status changes.
- To search and filter applications.
- To generate reports and counts.
- To provide an easy to use web interface.

Technology Stack:

- Python Flask
- SQLite
- HTML
- CSS

2. ER Diagram

The ER diagram illustrates the data model. It includes four tables: Student (with attributes student_id (PK), name, department, year, cgpa, email (Unique)), Company (with attributes company_id (PK), company_name, role, package, drive_date), Application (with attributes application_id (PK), student_id (FK), company_id (FK), applied_date), and StatusHistory (with attributes history_id (PK), application_id (FK), status, status_date). Relationships are shown as 1:N between Student and Application, and 1:N between Company and Application. A 1:1 relationship exists between Application and StatusHistory.

3. Architecture Diagram

The architecture diagram shows the system's components and their interactions. A User (Placement Officer / Student) interacts with a Web Browser (HTML Interface). The Web Browser connects to a Flask Application (Python Backend), which in turn connects to a SQLite Database (Stores Data). The database provides data for Search, Filter, Reports & Counts.

4. Database Tables

Column	Type	Constraints
student_id	INTEGER	PK
name	TEXT	NOT NULL
department	TEXT	NOT NULL
year	INTEGER	CHECK 1-4
cgpa	REAL	CHECK 0-10
email	TEXT	UNIQUE

Column	Type	Constraints
company_id	INTEGER	PK
company_name	TEXT	NOT NULL
role	TEXT	NOT NULL
package	REAL	CHECK > 0
drive_date	DATE	NOT NULL

Column	Type	Constraints
application_id	INTEGER	PK
student_id	INTEGER	FK
company_id	INTEGER	FK
applied_date	DATE	NOT NULL

UNIQUE (student_id, company_id)

Column	Type	Constraints
history_id	INTEGER	PK
application_id	INTEGER	FK
status	TEXT	CHECK
status_date	DATE	NOT NULL

5. Key Features

- ✓ Add new placement application
- ✓ View all applications
- ✓ Search by student name
- ✓ Filter by application status
- ✓ Track status history
- ✓ Record count
- ✓ Simple and responsive UI

6. SQL Queries (Examples)

- Display all applications
`SELECT * FROM Application ORDER BY applied_date DESC;`
- Search student
`SELECT * FROM Student WHERE name LIKE '%Kamalika%';`
- Filter shortlisted students
`SELECT * FROM StatusHistory WHERE status = 'Shortlisted';`
- Count applications
`SELECT COUNT(*) FROM Application;`

7. Sample Output Screens

(A) Placement Tracker

College Placement Drive and Student Application Tracker

Add Application View Applications Search Student

(B) Add Application

Student: Backlisting
Company: TCS
Applied Date: 01/08/2026
Save Application

(C) Application List

Total Records : 4

ID	Student	Company	Role	Status	Date
4	Vignesh	Wipro	Project Engineer	Applied	04/08/2026
3	Riya	Zoho	Software Developer	Interview	03/08/2026
2	Asika	Infosys	System Engineer	Applied	02/08/2026
1	Kamalika	TCS	Software Engineer	Shortlisted	01/08/2026

Filter by Status: Applied | Shortlisted | Interview | Selected | Rejected | Offered

(D) Search Student

Search Student: Kamalika
Search

Student	Company	Status
Kamalika	TCS	Shortlisted

8. Constraints Demonstration

✗ Duplicate Email
`INSERT INTO Student (name, department, year, cgpa, email) VALUES ('Rahul', 'IT', 2, 8.6, 'kamalika@gmail.com');`
Result: UNIQUE constraint failed

✗ Invalid CGPA
`INSERT INTO Student (name, department, year, cgpa, email) VALUES ('Rahul', 'IT', 2, 12, 'rahul@gmail.com');`
Result: CHECK constraint failed

✗ Duplicate Application
`INSERT INTO Application (student_id, company_id, applied_date) VALUES (1, 1, '2026-08-15');`
Result: UNIQUE constraint failed

✗ Invalid Student ID
`INSERT INTO Application (student_id, company_id, applied_date) VALUES (100, 1, '2026-08-20');`
Result: FOREIGN KEY constraint failed

✗ Negative Package
`INSERT INTO Company (company_name, role, package, drive_date) VALUES ('ABC', 'Tester', -5000, '2026-09-01');`
Result: CHECK constraint failed

9. File Structure

```
PlacementTracker/
├── app.py
├── requirements.txt
├── database/
│   ├── schema.sql
│   ├── seed.sql
│   └── templates/
│       ├── index.html
│       ├── add_application.html
│       ├── applications.html
│       └── search.html
├── static/
│   ├── style.css
│   └── screenshots/
│       ├── home.png
│       ├── add.png
│       ├── list.png
│       └── search.png
└── README.md
```

10. How to Run

- Install Python 3.x
- Install Flask
`pip install -r requirements.txt`
- Create Database
`sqlite3 database.db < database/schema.sql`
- Insert Sample Data
`sqlite3 database.db < database/seed.sql`
- Run the Application
`python app.py`
- Open in Browser
`http://127.0.0.1:5000`

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output

File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

IMPLEMENTATION

File Structure

Organized project layout for the PlacementTracker application.

```
PlacementTracker/  
  database/  
    schema.sql  
    seed.sql  
  templates/  
    home.html  
    add_application.html  
    applications.html  
    search.html  
  static/  
    style.css  
  app.py  
  requirements.txt  
  README.md
```

- app.py**
Main Flask application — routes, DB connection, request handlers.
- database/**
schema.sql defines all 4 tables; seed.sql inserts sample records for testing.
- templates/**
Jinja2 HTML templates for each view — home, add, list, and search.
- static/style.css**
All UI styling — layout, colors, table, forms, and responsive rules.
- requirements.txt**
Lists Python dependencies — Flask and any additional packages.

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output
- File Structure

SUMMARY

How to Run

- Future Scope
- Conclusion

SUMMARY

How to Run

Set up and launch the Placement Tracker locally in six steps.

- 1

Install Python 3.x

```
python --version
```
- 2

Install Flask

```
pip install -r requirements.txt
```
- 3

Create Database

```
sqlite3 database.db < database/schema.sql
```
- 4

Insert Sample Data

```
sqlite3 database.db < database/seed.sql
```
- 5

Run the Application

```
python app.py
```
- 6

Open in Browser

```
http://127.0.0.1:5000
```

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope
- Conclusion

SUMMARY

Future Scope

Planned enhancements to elevate the system into a production-grade placement platform.



Admin Login

Role-based access control for placement officers and students.



Email Notifications

Automated email alerts when application status changes.



SMS Alerts

SMS notifications for drive reminders and status updates.



Resume Upload

Allow students to upload and manage their resumes within the portal.



Dashboard Analytics

Visual charts showing placement rates, company-wise stats, and trends.



PDF Reports

Exportable PDF placement reports for college records and accreditation.

GETTING STARTED

- Introduction
- Problem Statement
- Objective

ARCHITECTURE

- System Comparison
- Tech Stack

IMPLEMENTATION

- Key Features
- SQL Queries
- Sample Output
- File Structure

SUMMARY

- How to Run
- Future Scope

Conclusion

SUMMARY

Conclusion

The Placement Tracking System delivers a complete digital solution for campus placement management.

- Successfully developed a Placement Tracking System using Python Flask and SQLite.
- Stores all placement records — students, companies, applications, and status history.
- Reduces manual work and eliminates errors through database constraints and automation.
- Easy to search and filter — by student name, company, or application status.
- Provides accurate placement reports for administrative and accreditation use.

```
return 200 OK // Thank You
```